

WaterOS

基于Rust面向Risc-V64和Loongarch64的Linux ABI宏内核

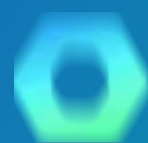
队伍：OuterSystems —— 山东大学（青岛）

队伍成员：宋浩宇 李佳灿 孙馨宇

指导老师：颜廷坤 潘润宇

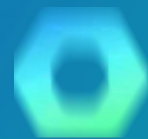
目录

CONTENTS



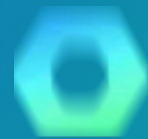
内核简介

我们实现了哪些功能



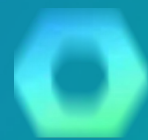
系统架构

我们系统的整体结构



模块设计

不同组件的具体实现



借鉴与创新

总结与未来发展计划

01

KernelOverview

内核简介

内核简介

Kernel Overview

WaterOS是Rust从零构建的双架构操作系统内核，支持RISC-V64与LoongArch64，兼容Linux ABI，涵盖进程调度、内存管理、VFS文件系统、网络协议栈与VirtIO驱动等子系统。

WATER OS



KernelOverview内核简介

硬件支持



Qemu 全支持

WaterOS可以在qemu虚拟机上使用全部已经实现的功能



开发板部分移植

WaterOS可以在Vision Five 2开发板和Loongson 2K1000开发板正常引导，启动，读写文件系统，运行busybox用户程序。

软件支持



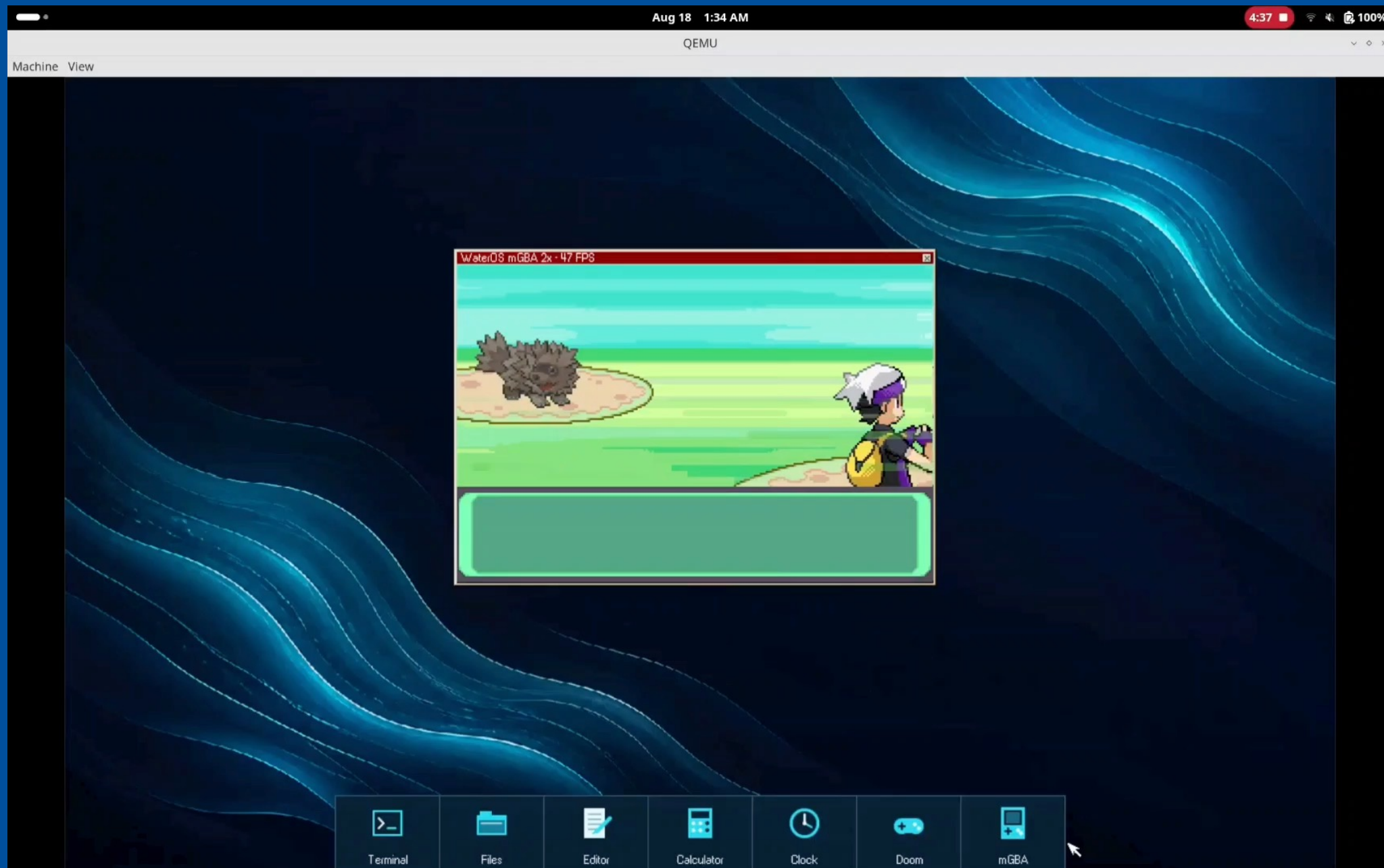
包管理器支持

可以使用APT和PACMAN包管理器安装构建软件包，如Neovim



Nanox支持

经测试，可以运行nanox桌面窗口环境



运行apt包管理器安装vim

```

VIM - Vi IMproved

        version 9.2.600
        by Bram Moolenaar et al.
Vim is open source and freely distributable


        Sponsor Vim development!
type  :help sponsor<Enter>    for information

type  :q<Enter>               to exit
type  :help<Enter> or <F1>    for on-line help
type  :help version9<Enter>  for version info

```

02

KernelOverview

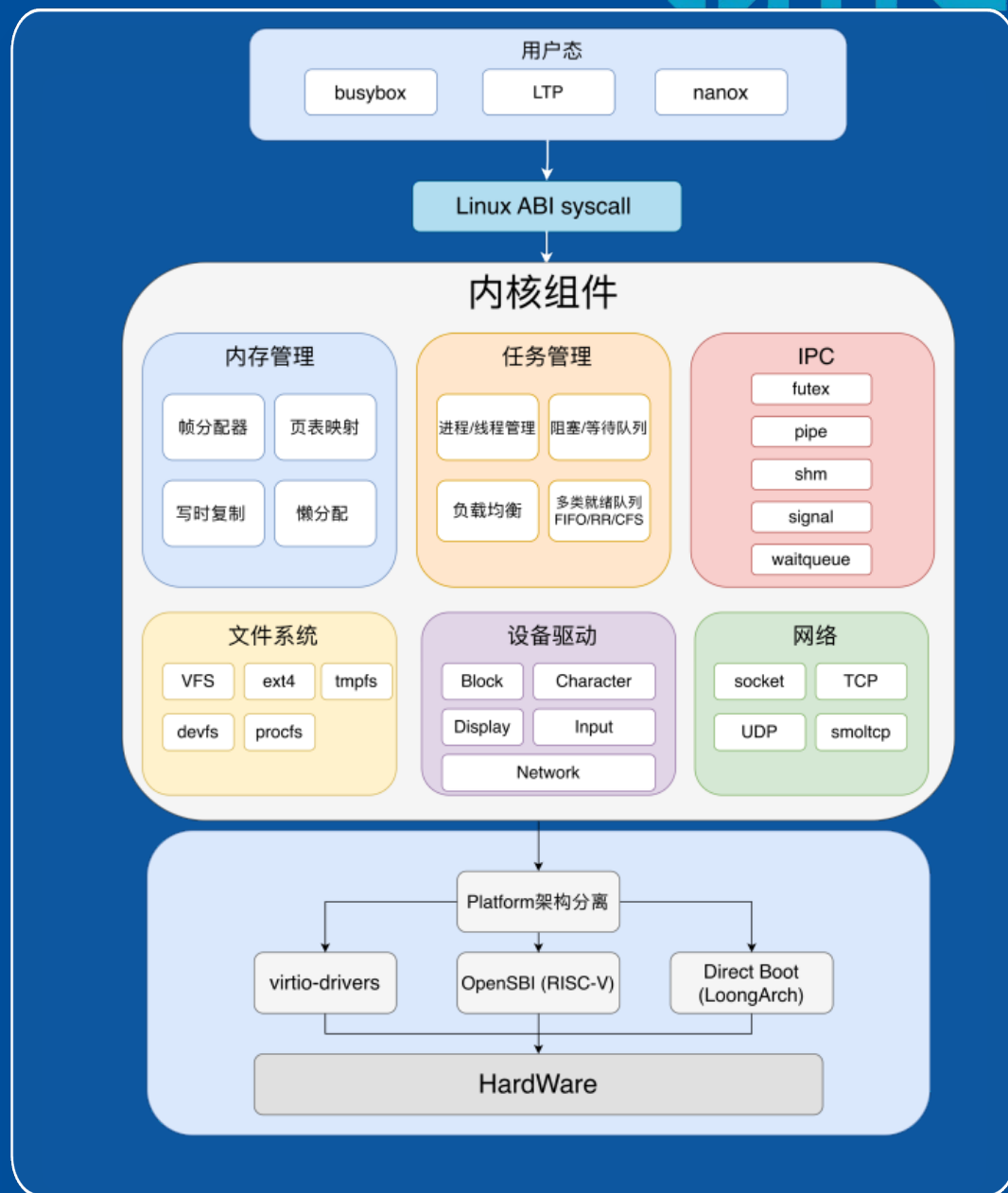
系统架构

System Architecture / 系统架构

WaterOS 内核概述

WaterOS是Rust从零构建的双架构操作系统内核，支持RISC-V64与LoongArch64，兼容Linux ABI，涵盖进程调度、内存管理、VFS文件系统、网络协议栈与VirtIO驱动等子系统。

- 双架构支持 (RISC-V64 / LoongArch64)
- Linux ABI 完全兼容
- Rust 语言从零安全构建



03

KernelOverview

模块设计

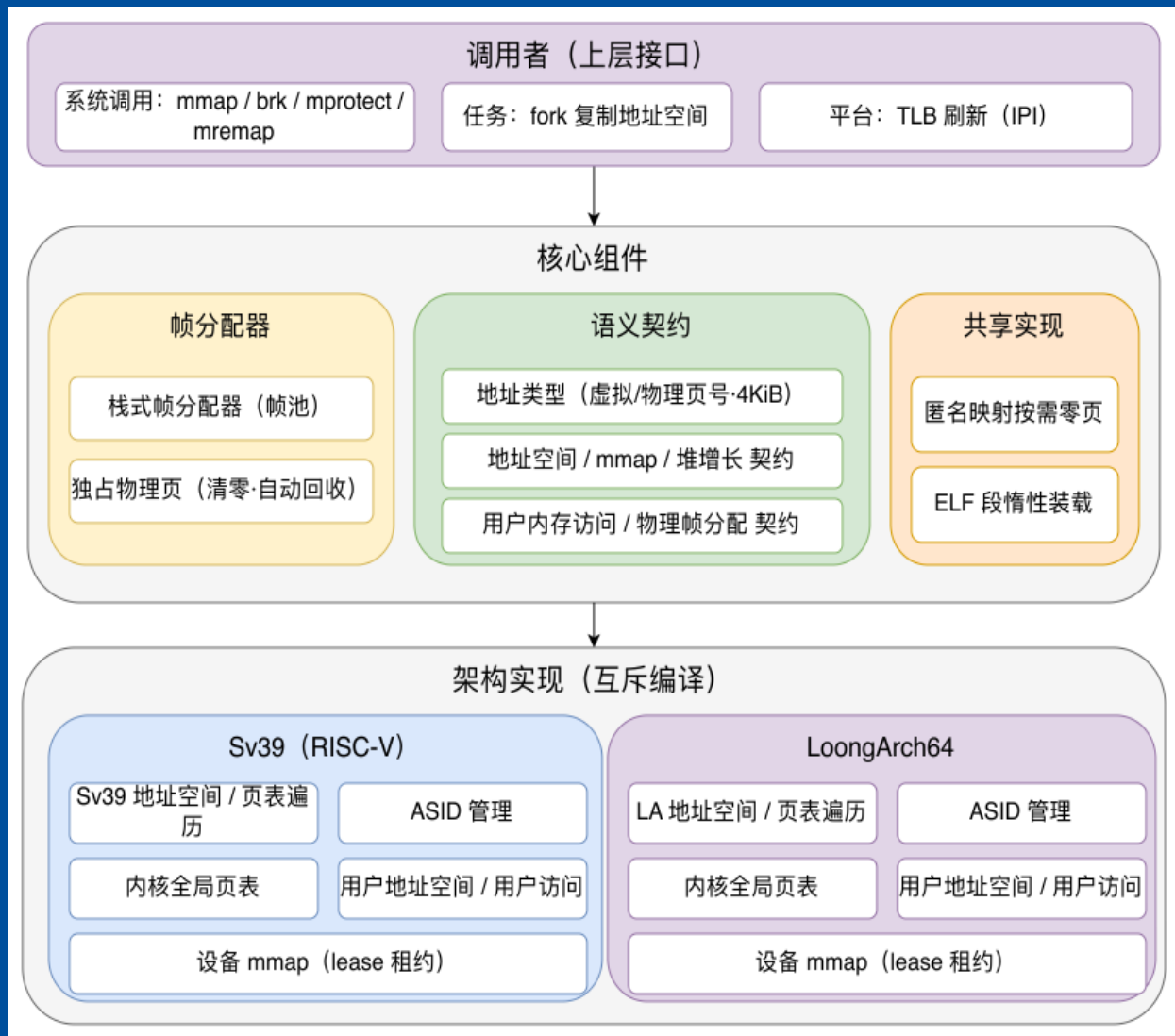
WaterOS-Memory: 内存模块

! 核心映射机制

内存管理将物理内存按 **4 KiB 页** 租给各进程的虚拟地址空间，核心是「**虚拟地址 → 页表 → 物理帧**」的映射。上层用 mmap/brk 等分配调整映射，fork 靠**写时复制 (COW)** 省复制，改页表后发 IPI 刷 TLB。

三大核心组件

- **栈式帧分配器**：管理物理帧，提供独占物理页与自动回收。
- **语义契约**：定义地址类型与统一接口，规范地址空间行为。
- **共享实现**：提供按需零页和 ELF 惰性装载，访问缺页才真正分配物理帧。



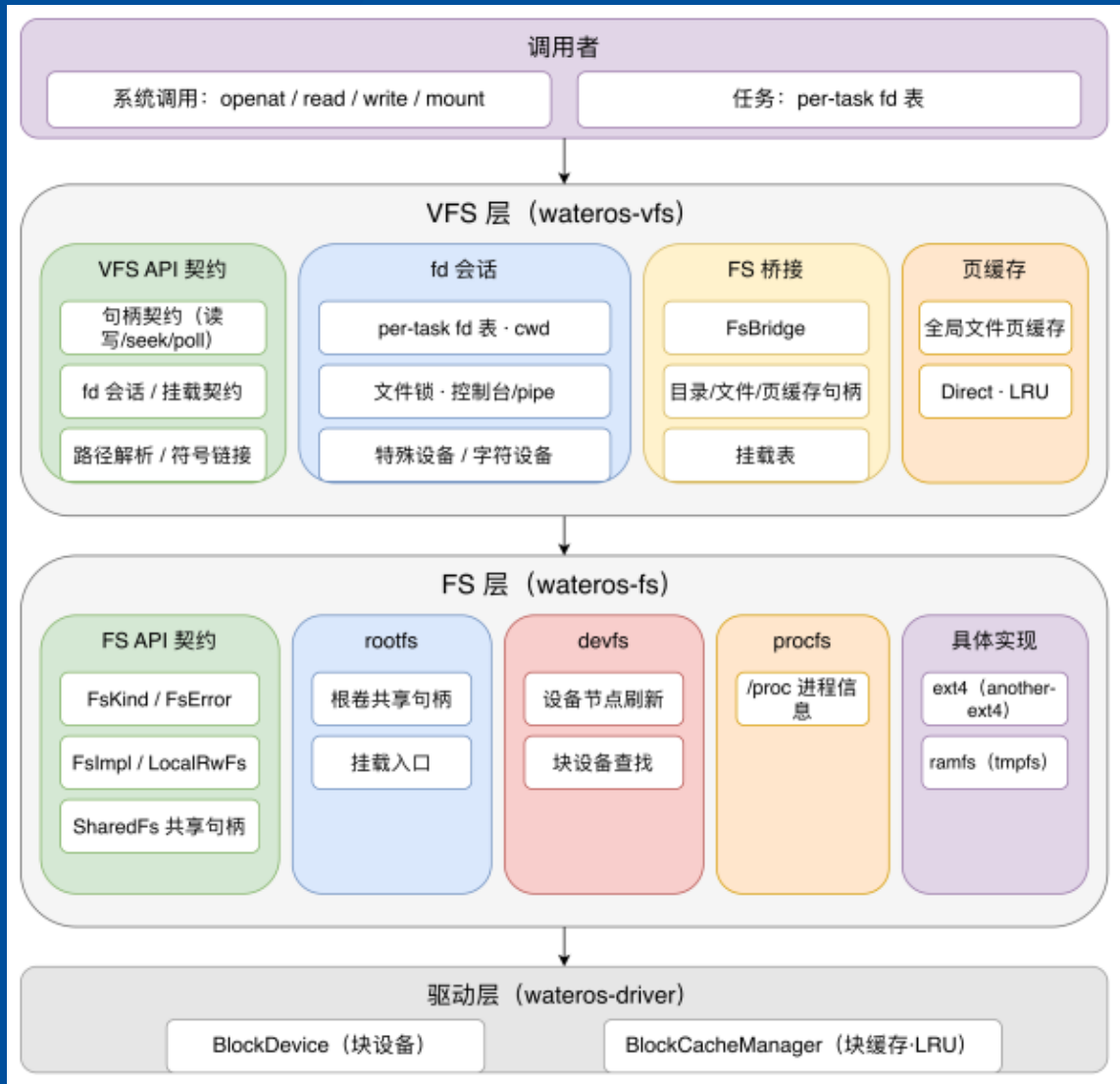
WaterOS-FS/VFS: 文件系统模块

统一文件访问

提供 open/read/write/mount 等标准文件操作，支持路径解析、符号链接以及文件与目录管理。为每个进程独立维护文件描述符和工作目录，并支持管道、控制台及设备文件访问。

多类型文件系统支持

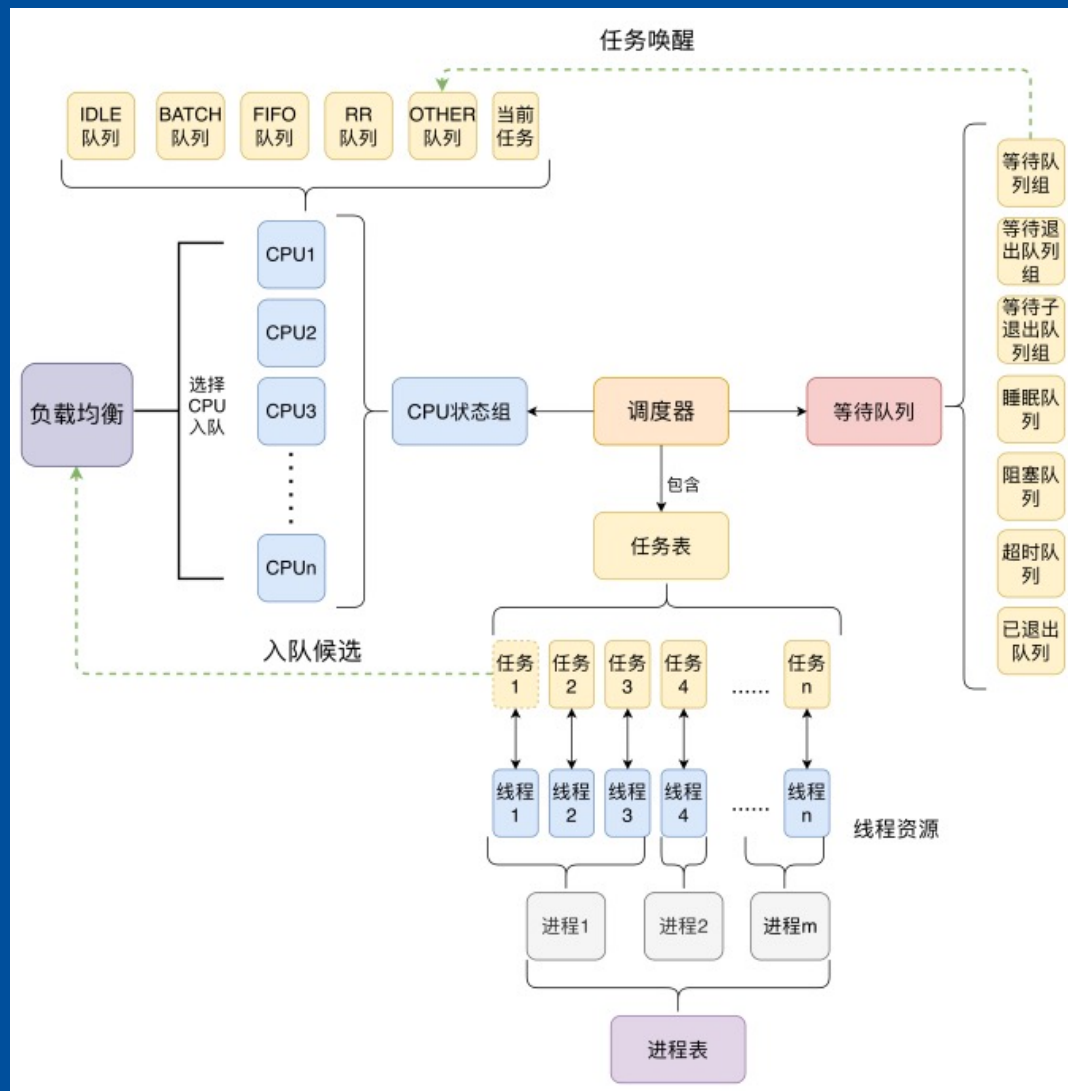
支持 ext4、rootfs、ramfs/tmpfs、devfs 与 procfs，满足持久化存储、内存文件、设备访问和系统信息查询需求。通过页缓存与块缓存减少磁盘访问开销，并支持文件系统挂载、卸载及多文件系统协同工作。



WaterOS-Task：任务模块

整体架构：

- **任务 (task)**：是调度实体，即调度器进行调度的基本单位，维护了调度状态和任务资源等属性。
- **进程和线程**：是一对多的关系，进程中存放有线程表，维护了地址空间、资源限制、进程能力等资源。
- **调度器**：负责 CPU 状态维护与任务调度管理，维护了各个CPU 的资源：就绪队列与当前任务 Cache 等。实现了五种调度方式以及基于入队的负载均衡策略，实现了多核多策略调度。

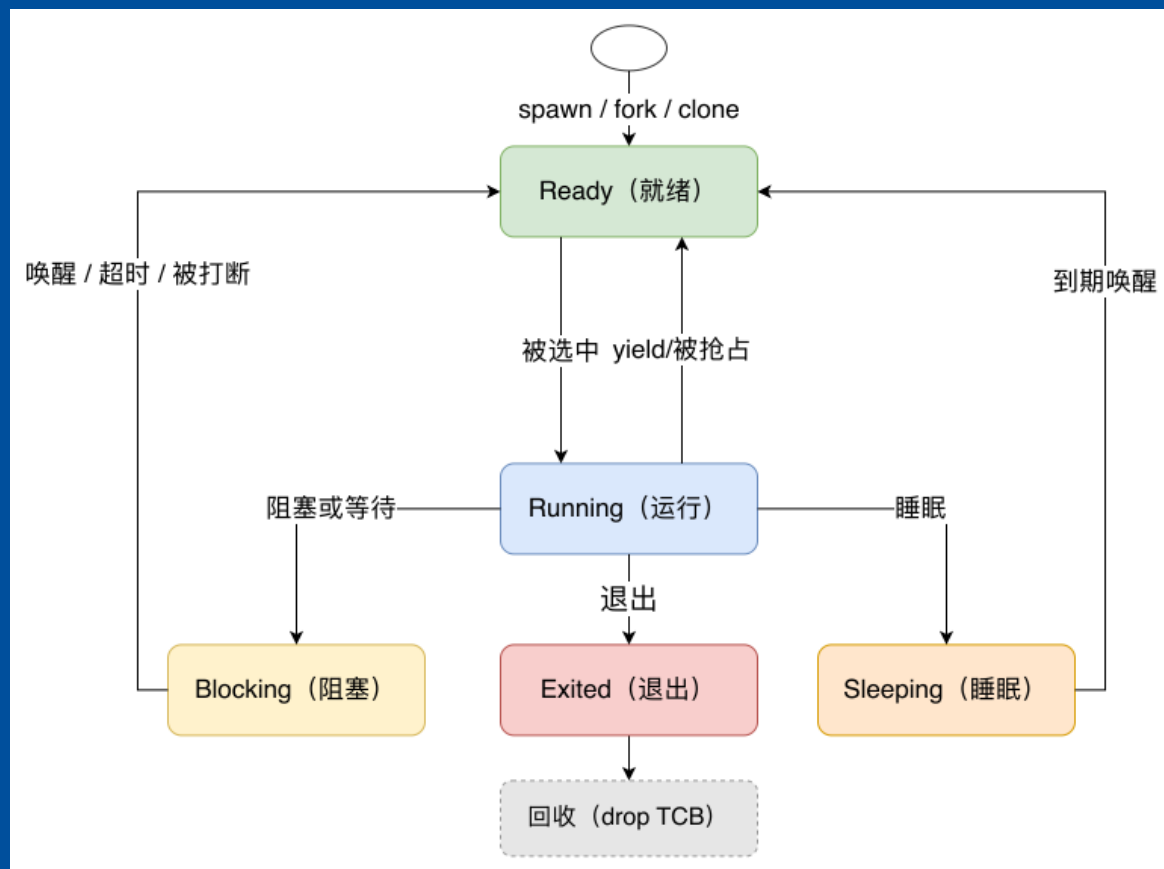


WaterOS-Task: 任务模块

任务状态机:

任务从创建到回收，一生在就绪（Ready）→ 运行（Running）→ 阻塞/睡眠/退出之间迁移。

- Ready (就绪): 挂在对应就绪队列里等调度
- Running (运行): 正占用 CPU 执行
- Blocking (阻塞): 等待某事件到达后回 Ready
- Sleeping (睡眠): 主动睡眠指定 tick 数，到期唤醒
- Exited (退出): 已终止，等待父进程回收



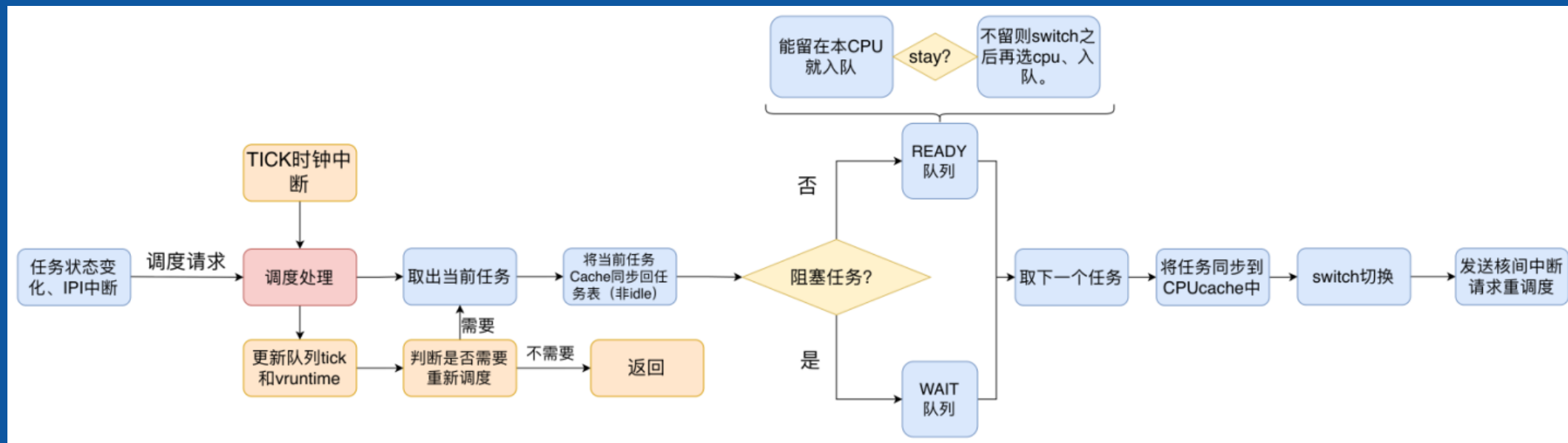
WaterOS-Task: 任务模块

● 调度逻辑

调度器以「触发 → 处理 → 取出入队 → 选下一个 → 切换 → 通知」为一个闭环，所有路径最终都汇到 `__switch` 切换上下文，再向远端 CPU 发 IPI 请求重调度。

● 触发源说明

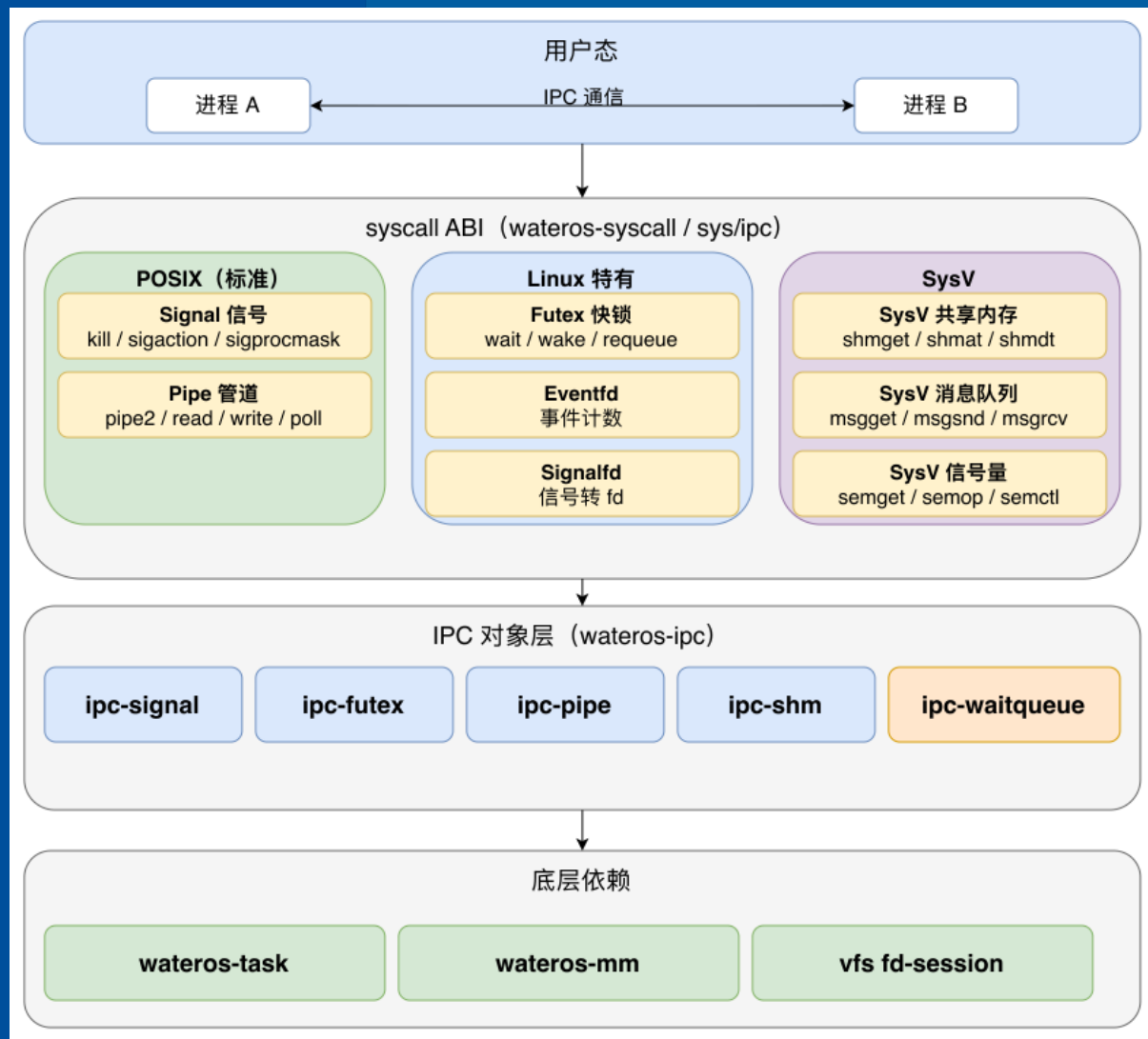
- ✓ **任务状态变化：**任务主动让出、阻塞、睡眠或退出
- ✓ **IPI 中断：**远端 CPU 发来的重调度请求
- ✓ **TICK 时钟中断：**周期性推进 tick，检查是否该切换



WaterOS-IPC: 进程通信模块

IPC 让隔离的进程能交换数据、同步进度。统一挂到 ipc-waitqueue 上，由调度器完成阻塞唤醒。

- **Signal:** 异步通知，维护 pending/mask/disposition
- **Futex:** 用户态锁阻塞与唤醒，支持 wait/wake/requeue
- **Eventfd:** 事件计数，读清零，用于通知唤醒
- **Pipe:** 字节流传输，单向通信
- **消息队列:** 有边界消息收发
- **共享内存:** 物理帧映射进双方地址空间
- **信号量:** 进程间同步与互斥



WaterOS-Network: 网络模块

01

SocketRef

封装底层协议栈句柄，统一接口；接入 VFS 的 FD 表复用读写接口。

02

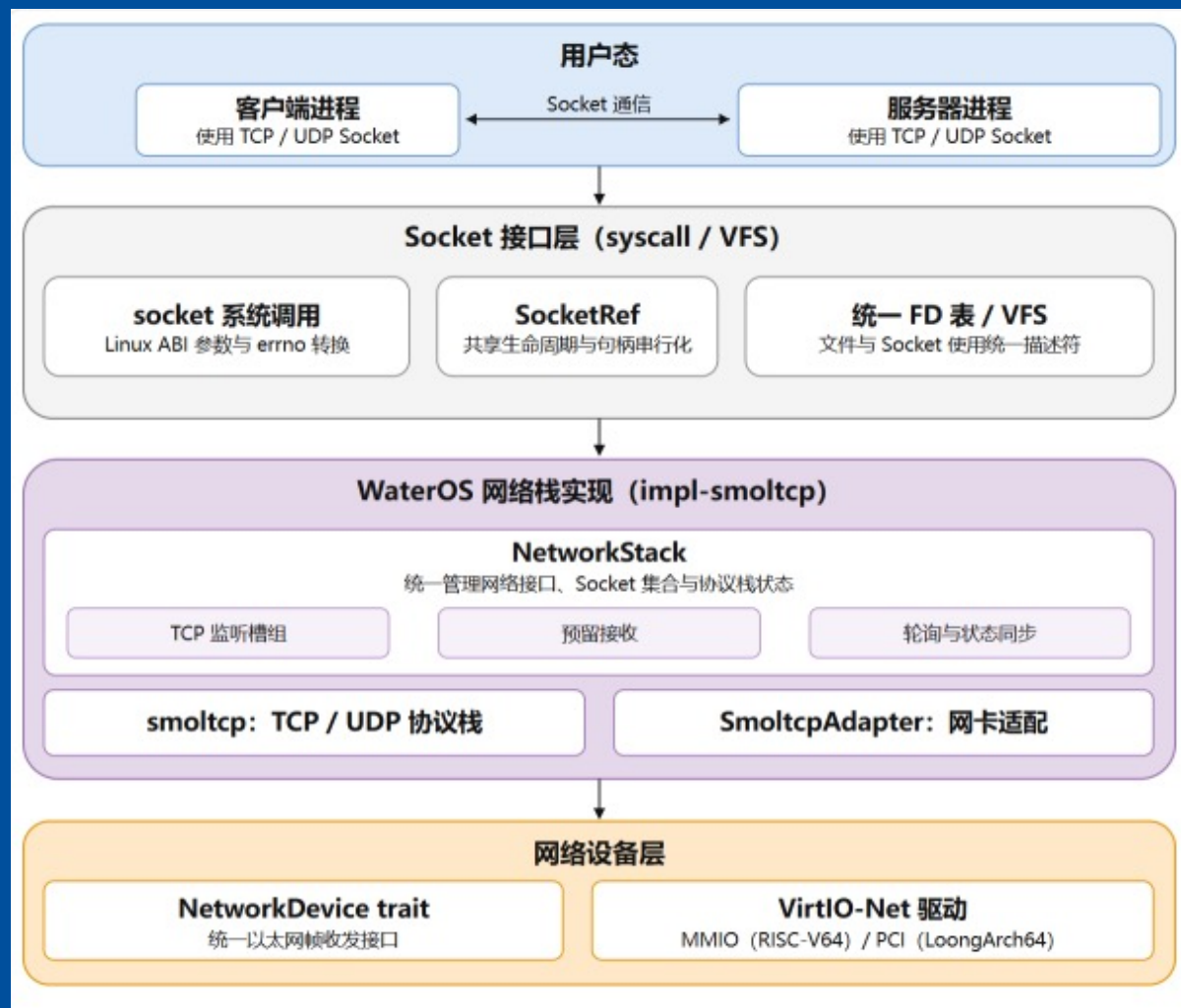
网络栈实现

基于 smoltcp 提供 TCP/UDP；NetworkStack 维护连接状态和 Linux Socket 语义。

03

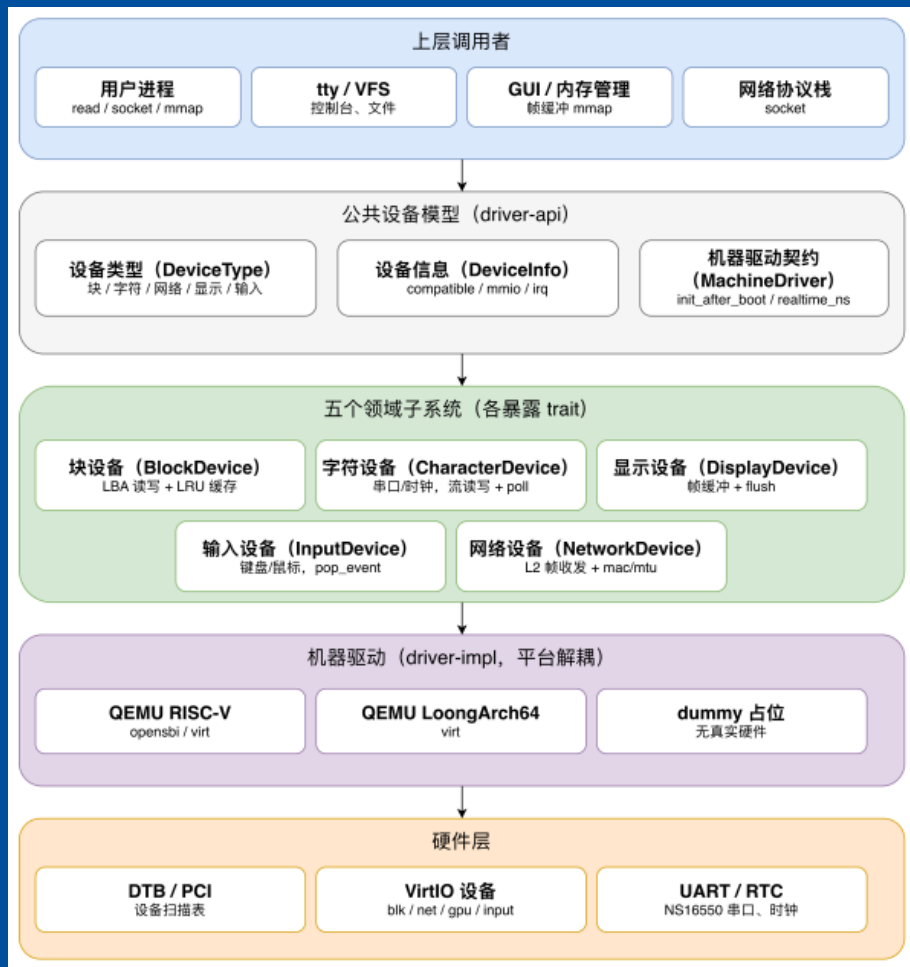
设备抽象

通过 NetworkDevice trait 统一接入 VirtIO-Net MMIO 与 PCI 网卡，解耦驱动。



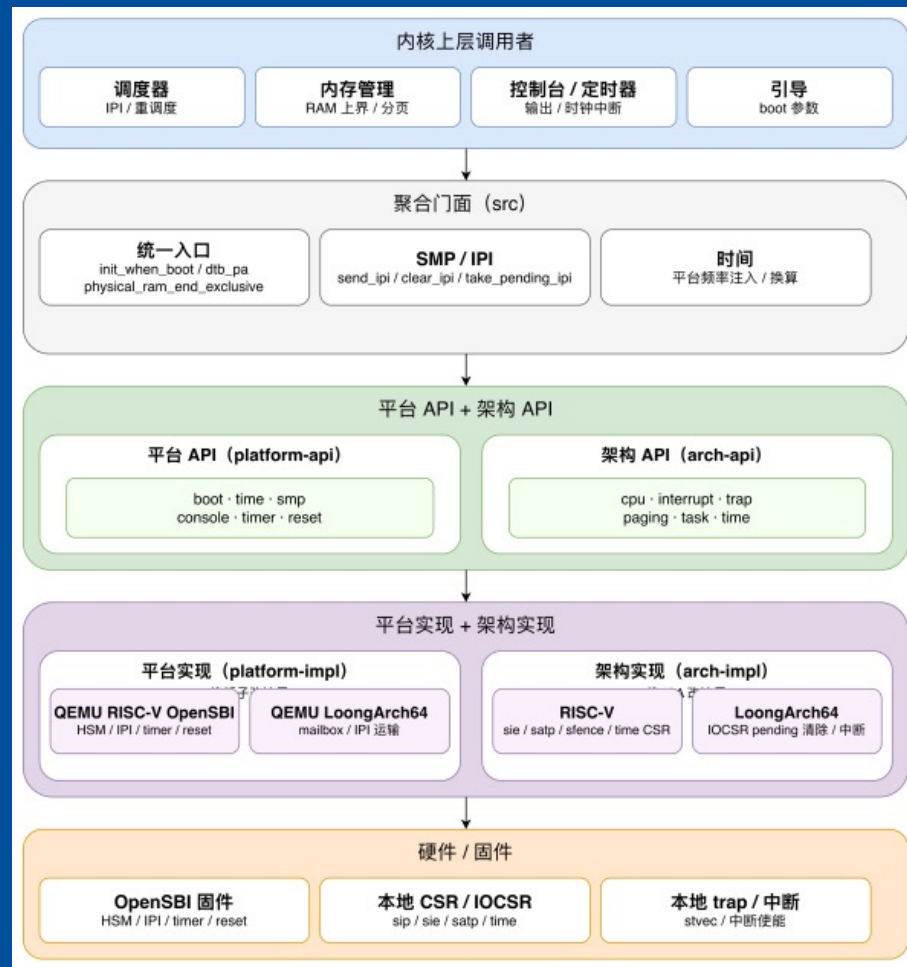
WaterOS-Driver/Platform: 驱动/平台模块

Driver架构



统一块设备、字符设备、显示、输入与网络设备接口，降低内核上层对具体硬件的依赖。

Platform架构



抽象启动、时钟、中断、SMP与内存等平台能力，为内核提供统一调用入口。

04

KernelOverview

借鉴与增量
工作

借鉴 Linux 机制与开源生态整合

借鉴 Linux 核心机制

系统调用与进程

对齐 Linux generic 64-bit 系统调用 ABI，借鉴进程/线程模型、clone、fork+COW 机制。

内存与文件

借鉴 VMA + page cache 思路，mmap 懒映射；融合 Linux VFS / dcache，使用 page cache 与块缓存。

调度与通信

支持 SCHED_OTHER/FIFO 等五类调度；引入 futex、rt_sigaction、pipe 语义。

增量工作

组件复用

网络（smoltcp）、文件系统（ext4_rs/another_ext4）、驱动（virtio-drivers）等，并基于 BusyBox 和 Nano-X 构建用户空间。

深度定制与修复

为 virtio-drivers 扩展区域刷新，修复 ext4 块分配与链接问题，并自研 SmoltcpAdapter 桥接网络栈与全局套接字池。

内核架构工程与机制创新

架构与工程创新

- 1 **Cargo Feature 驱动组合**: 利用编译期 feature 树实现“平台×阶段×能力×实现”的灵活组合，一套源码产出多环境内核，边界清晰且精简。
- 2 **双架构共享核心**: RISC-V64 与 LoongArch64 的底层差异（启动/页表/中断）收敛于平台层，上层模块全面复用共享。
- 3 **API + Impl 模块设计**: 定义统一 Trait 与语义契约，不同实现按需挂载。

内核机制演进

- 1 **统一租约式 I/O 模型**: VFS、Socket 等统一采用“预约分配 → 复制确认 → 提交消费”两阶段模型，避免跨页 fault 丢数据。
- 2 **缓存准入与多层分治**: 实现文件页、块缓存与 ELF 缓存分离，引入“二次命中 ghost history”准入策略，防止一次性读取污染缓存。
- 3 **多策略调度与 SMP 均衡**: 实现五类调度类结合 CFS 的 vruntime，配合每 CPU 独立运行队列与定向 IPI 完成负载均衡。

05

KernelOverview

总结

Work Summary / 项目总结与测试

完成状况

当前内核已完全实现**双架构支持**、**多核调度**、虚拟内存（按需分页 + COW）、多文件系统、完整 IPC 机制、网络栈、设备驱动与图形桌面环境。

验证成果

已通过 LTP (Linux Test Project)、stress-ng 压力测试、Imbench 性能基准测试以及 busybox 等业界标准工具链的严格验证，具备**良好的兼容性与稳定性**。

未来发展计划： 面向多核高负载与真实设备场景，重点提升资源隔离、回收可靠性和系统可观测性。

调度与内存

实现忙核主动均衡及自适应调度策略；支持用户栈向下动态增长。

进程管控

引入 cgroup 资源限制、完整的 capability 安全机制及进程记账 (accounting)。

系统稳定性

优化 fork 高频并发场景下的内存资源快速回收。

设备与文件系统

完善设备模型及热插拔支持；补全 procfs/sysfs 真实硬件中断与网络统计数据。

感谢您的聆听

学无止境 气有浩然